Project 1 — E-Commerce Sales Intelligence Dashboard

DAY 4

Author: Sudiksha Gunjkar | Tool: DuckDB 1.5.3 + Python | Date: 07 June 2026

Objective

Write 3 production-grade analytical SQL queries using DuckDB against the processed star-schema tables. Each query demonstrates a senior-level SQL pattern: window functions (rolling aggregations), cohort analysis (date-based self-join), and RFM segmentation (NTILE scoring + CASE WHEN labels). All query results saved as CSVs — ready to load directly into Power BI in Week 2.

Tool used — DuckDB

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Property' 'frags': [ParaFrag (__tag__='b', bold=1, fontName ='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Property', te xtColor=Color(1,1 ,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Detail' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Detail', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph

Tool	DuckDB 1.5.3 (in-process analytical SQL engine)
Why DuckDB	Reads CSVs directly without import step. 10x faster than SQLite for analytical queries. Supports full window functions.
Connection	duckdb.connect() — in-memory, no server needed
Input	6 CSVs from data/processed/ loaded as SQL tables via read_csv_auto()
Output	3 .sql files + 3 result CSVs saved to sql/
Runtime	All 3 queries completed in under 5 seconds

Query 1 — Rolling 30-Day Revenue

Purpose:
Track smoothed revenue trend removing day-to-day noise. Rolling averages are a core BI KPI — this query powers a rolling avg card in Power BI.

Key SQL patterns used:

SQL pattern	What it computes
`SUM() OVER (ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)`	Rolling 30-day revenue — sums the current day + 29 previous days
`SUM() OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`	Rolling 7-day revenue — shorter window for weekly trend
`SUM() OVER (ROWS UNBOUNDED PRECEDING)`	Cumulative running total revenue from the first day
`LAG(daily_revenue) OVER (ORDER BY order_date)`	Previous day's revenue — used to compute day-over-day change %

Result: 774 rows (one per calendar day) | Max rolling 30d revenue: confirmed from output

Output columns: order_date, daily_revenue, rolling_7d_revenue, rolling_30d_revenue, cumulative_revenue, dod_change_pct

Files saved: sql/01_rolling_revenue.sql + sql/result_01_rolling_revenue.csv

Query 2 — Customer Cohort Analysis

Purpose:
Group customers by their first purchase month (cohort). Track how many return in subsequent months to reveal true retention rate. This goes deeper than the 97% one-time buyer stat found in EDA.

Key SQL patterns used:

SQL pattern	What it computes
`DATE_TRUNC('month', date)`	Truncates a date to the first day of its month — groups orders into monthly cohorts
`MIN(date) GROUP BY customer_key`	Finds each customer's very first order date — their acquisition date
`DATEDIFF('month', cohort_month, order_month)`	Calculates how many months since a customer's first purchase (0 = acquisition month)
`active_customers * 100.0 / cohort_size`	Retention rate % — what fraction of the original cohort is still active

Key finding: Month-1 retention is extremely low — consistent with the 97% one-time buyer finding from EDA

Output columns: cohort_month, cohort_size, order_month, months_since_first, active_customers, retention_rate_pct

Files saved: sql/02_cohort.sql + sql/result_02_cohort.csv

Query 3 — RFM Customer Segmentation

Purpose:
Score every customer on 3 dimensions: Recency (how recently they ordered), Frequency (how often), and Monetary value (how much they spent). Assign each customer to 1 of 8 business segments. This CSV loads directly into Power BI as a customer segmentation slicer.

Key SQL patterns used:

SQL pattern	What it computes
`NTILE(4) OVER (ORDER BY recency_days DESC)`	Splits customers into 4 equal buckets by recency. Score 4 = most recent (best).
`NTILE(4) OVER (ORDER BY frequency ASC)`	Splits by order frequency. Score 4 = highest frequency (best).
`NTILE(4) OVER (ORDER BY monetary ASC)`	Splits by total spend. Score 4 = highest spend (best).
`CASE WHEN r=4 AND f=4 AND m=4 THEN 'Champions'...`	Translates numeric scores into business segment labels for non-technical users.
`DATEDIFF('day', MAX(order_date), snapshot_date)`	Recency = days between customer's last order and the dataset's last date.

RFM Segment Distribution (from terminal output):

Segment	Count	%	Business meaning
Hibernating	~65,000+	~65%+	Low recency, low frequency — largest group, likely one-time buyers
Need Attention	13,140	13.2%	Mid-tier customers slipping away — need re-engagement campaigns
At Risk	4,381	4.4%	Previously good customers not ordering recently — churn risk
Champions	1,128	1.1%	Best customers — high R, F, M scores — protect and reward
Recent Customers	~est.	~%	Ordered recently but not yet frequent — nurture into loyalty
Potential Loyalists	~est.	~%	Recent + decent spend — upsell opportunity
Loyal Customers	~est.	~%	High frequency — may not be highest spend but very engaged
Cannot Lose Them	~est.	~%	High value but not ordering recently — win-back priority

Key finding: Only 1.1% Champions (1,128 customers) out of 99,441 — massive loyalty gap vs best-in-class e-commerce

Key finding: 4.4% At Risk customers (4,381) = revenue at immediate risk — win-back campaign priority

Files saved: sql/03_rfm.sql + sql/result_03_rfm.csv

Files produced today

File	Description
`day4_sql_queries.py`	Main script — loads tables, runs 3 queries, saves results
`sql/01_rolling_revenue.sql`	Rolling 30-day revenue SQL query
`sql/result_01_rolling_revenue.csv`	Query 1 result — 774 rows, daily + rolling revenue
`sql/02_cohort.sql`	Customer cohort analysis SQL query

`sql/result_02_cohort.csv`	Query 2 result — retention by cohort month
`sql/03_rfm.sql`	RFM segmentation SQL query
`sql/result_03_rfm.csv`	Query 3 result — 99,441 customers with RFM scores + segment

Key learnings from Day 4

1. DuckDB's read_csv_auto() loads CSVs directly as SQL tables — no import step, no schema definition needed.

2. NTILE(4) is the correct window function for RFM scoring — splits rows into equal quartile buckets.

3. Window functions (ROWS BETWEEN n PRECEDING AND CURRENT ROW) require ORDER BY inside the OVER clause — without it the frame is undefined.

4. NULLIF(denominator, 0) prevents division-by-zero errors in percentage calculations — essential for production SQL.

5. DATE_TRUNC is cleaner than stripping year+month separately for cohort grouping — more readable and portable.

6. Only 1.1% Champions in the dataset is a powerful business insight — most e-commerce platforms target 5-10%.

7. RFM segment labels in plain English (Champions, At Risk) make SQL output immediately usable by business teams without a data analyst translating.

Day 5 — next steps

Task	Detail
Create GitHub repo	Name: ecommerce-sales-dashboard | Public repo
Add .gitignore	Exclude data/raw/ (large CSVs), venv/, __pycache__/
Write README.md	Project goal, tech stack, schema screenshot, 3 key findings with real numbers
Push all code files	etl_pipeline.py, notebooks/*.py, sql/*.sql, .gitignore
Add charts folder	Push notebooks/charts/ — 15 PNG files visible on GitHub
Verify on GitHub	Check README renders correctly, folder structure is clean